

Das OSG Lecture Bundle

Benutzerhandbuch

Matthias Werner

24. August 2026

Inhaltsverzeichnis

1 Einführung	1
1.1 Polymorphie in OSG Lecture	2
2 Konzepte	2
2.1 Projekt	2
2.2 Lehreinheit	3
2.3 Verzeichnisstruktur	3

1 Einführung

Lehrmaterialien treten in vielerlei Formen und Versionen auf, beispielsweise als Vorlesungsskript, Folien oder Handouts, und das möglicherweise jeweils in verschiedenen Sprachen. Dabei sind die Inhalte oft gleich oder ähnlich, und die Strukturen – beispielsweise die Reihenfolge der Lehr- und Lerneinheiten sollten konsistent sein. Mit anderen Worten: Die gleiche Sache (hier: semantische Information) soll in unterschiedlicher Gestalt auftreten. In der Informatik oder in der Chemie nennt man dieses Konzept auch *Polymorphie*¹.

Gleichzeitig werden Lehrmaterialien in der Regel permanent überarbeitet. Einzelne Lehreinheiten werden modernisiert, reorganisiert oder gleich ganz entfernt, neue werden ergänzt. Dabei stets die gewünschte Konsistenz aufrechtzuerhalten, ist schwierig. Das OSG Lecture Bundle versucht, Lehrende dabei zu unterstützen. Es folgt dabei einer zentralen Idee:

¹Vielleicht wären *Polyrepräsentation* oder sogar *Multimodalität* noch passendere Begriffe, aber ersteres ist nicht sehr etabliert und beim zweiten müsste ich vermutlich mit Kulturwissenschaftlern und Semiotikern darüber diskutieren, ob unterschiedliche Sprachen als *Modi* aufzufassen sind. Außerdem bin ich Informatiker, da liegt die Polymorphie nahe.

Es sollten so **wenig** wie möglich **Redundanzen** auftreten.

Wo Redundanzen notwendig sind (beispielsweise bei verschiedenen Textversionen), ist die Aufrechterhaltung von Konsistenz einfacher, wenn aufeinander bezogenes Quellmaterial **möglichst nahe** beieinander ist.

Daher soll **OSG Lecture** u. a. ermöglichen :

- Generierung verschiedener Dokumentarten aus der gleichen Quelle;
- einfache Reorganisation von Lehreinheiten;
- verschiedene Sprachvarianten aus der gleichen Quelle, wobei gemeinsame Daten nicht redundant dargestellt werden brauchen;
- Unterstützung bei individuellen Erstellen und Überarbeiten von Lehreinheiten mit einfacher Integration (z. B. Skriptkapitel zu einem Gesamtskript);
- schnelle Bereitstellung der verschiedenen Dokumentarten.

Dabei wurde von Modalitäten und Arbeitsabläufen ausgegangen, die an der Professur für Betriebssysteme (*Operating Systems Group*, *OSG*) der TU Chemnitz derzeit üblich sind. Wenn Sie nicht dort tätig sind und z. B. andere Abläufe haben, könnte **OSG Lecture** trotzdem für Sie nützlich sein: einzelne Pakete des Bundles lassen sich auch unabhängig von den anderen nutzen.

OSG Lecture ist das Nachfolgeprojekt von **OSGBeamer**², das sich aus einem Thema und einer handvoll Ergänzungsmacros für die überaus verbreitete Beamer-Klasse³ entwickelt hat.

1.1 Polymorphie in **OSG Lecture**

Es ist eine alte $\text{T}_{\text{E}}\text{X}$ -Idee, Texte je nach Kontext unterschiedlich darzustellen. Die $\text{T}_{\text{E}}\text{X}$ -Variante $\text{ConT}_{\text{E}}\text{Xt}$ leitet beispielsweise ihren Namen auch davon ab. Die folgenden Beispiele zeigen den Einsatz in **OSG Lecture**.

2 Konzepte

2.1 Projekt

Als ein Projekt wird hier die Sammlung von zusammengehörigen Materialien verstanden. Im Kontext unsewres Anwendungsfalls bedeutet dies Lehrmaterialien wie Vorlesungsfolien

²<https://github.com/tuc-osg/osgbeamer>

³<https://ctan.org/pkg/beamer>

oder Skripts, auch wenn [OSG Lecture](#) auf andere Anwendungsfälle übertragbar ist. Mit anderen Worten: Ein [OSG Lecture](#)-Projekt ist ein *Kurse*.

Alles Kursmaterialien stehen in einem Verzeichnis, welches in der Regel ein spezielles Manifest enthält. Die Ausnahme zu dieser Regel sind sogenannte *standalone*-Projekte, ursprünglich für einen der oben angesprochenen anderen Anwendungsfälle gedacht waren, nämlich Konferenzbeiträge. Dieser Spezialfall wird im Abschnitt ?? betrachtet. Im Folgendem gehen wir aber von Kursen aus.

2.2 Lehreinheit

Ein Kurs ist aus Sicht von [OSG Lecture](#) eine Serie von zusammenhängenden Lehreinheiten, z. B. Vorlesungen oder Übungen (oder beidem), die typischerweise im Laufe eines Semesters gehalten werden. Eine Lehreinheit ist hier eher ein didaktisches Konzept als ein zeitliches Konzept: Es ist durchaus möglich, dass sich eine Lehreinheit über mehrere Termine streckt oder dass zu einem Termin mehrere (eher kurze) Einheiten behandelt werden. Welchen Satz von Lehrmaterialien für einen Kurs benötigt werden, kann recht unterschiedlich sein. In der OSG gibt es einige typische Lehrszenarien:

1. Es gibt Präsentationen, die die Lehrenden benutzen, während die Studierenden dazu passende Handouts erhalten.
2. Wie 1, aber die Handouts gibt es in einer englischen und einer deutschen Sprachfassung.
3. Wie 2, aber statt der Handouts pro Lehrthema gibt es jeweils ein Gesamtskript pro Sprache.

All diese und noch viel mehr Szenarien werden durch [OSG Lecture](#) unterstützt. Dabei spielt @1 die Rolle einer *Metaklasse*: Sie organisiert im Wesentlichen, welche konkrete Klasse geladen wird (z. B. `article`, `book`, `ltx-talk` oder `beamer`) und sorgt dafür, dass es ein gemeinsames Interface gibt.

2.3 Verzeichnisstruktur

Die Materialien jedes Kurs' sind in einem eigenen Verzeichnis, dem sogenannten (Kurs-)Projektverzeichnis oder (bezogen auf das Projekt) Rootverzeichnis. Dabei bildet die Verzeichnisstruktur die Stuktur des Kurses nach.

Im Folgenden ist eine mögliche Verzeichnisstruktur für ein einfaches Szenario zu sehen:

```
Operating_Systems/ ..... Root-Verzeichnis
├─ ollmconfig.toml ..... Manifest
├─ 000-Intro/ ..... 1. Lehreinheit
│
```

```

├── main.tex
├── picture.png
├── :
├── 010-Design/ ..... 2. Lehreinheit
│   ├── main.tex
├── 020-Process/ ..... 3. Lehreinheit
│   ├── main.tex
├── :
├── Include/ ..... gemeinsame Daten
│   ├── projectconfig.tex ..... Kurs-Metadaten
└── ...

```

Der Verzeichnisname einer Lehreinheit besteht mindestens aus einer dreistelligen Nummer und einer durch einen Bindestrich abgetrennten Themenbeschreibung. Die Nummer muss dreistellig sein (führende Nullen sind erlaubt) und legt lediglich die Reihenfolge der Lehreinheiten fest, nicht etwa Kapitelnummern. Daher bietet es sich mindestens in der Entwicklungsphase eines Kurses an, eine größere Schrittweite zu wählen, um später Kapitel ohne großen Aufwand umorganisieren zu können.⁴

Die Themenbeschreibung im Verzeichnisnamen ist frei wählbar und soll nur als Gedächtnisstütze dienen, was der Inhalt der jeweiligen Einheit ist. Mehrere Worte können durch Unterstrich oder Gedankenstrich getrennt werden, aber in der Regel empfiehlt es sich, ein einzelnes aussagekräftiges Wort zu nutzen.

Der Verzeichnisname kann aber noch zusätzliche Informationen beinhalten. Insgesamt hat der Verzeichnisname einer Lehreinheit das folgende Format:

`<nummer>[<target>]-[<role>]-<topic>`

Die Teile in eckigen Klammern sind jeweils optional. Sie haben folgende Bedeutung:

target Beschreibt das Ziel dieser Einheit.

Manchmal gibt es Material, das nur für bestimmte Arten von Materialien sinnvoll ist. Beispielsweise werden häufig die organisatorischen Inhalte der Auftaktvorlesung nicht mit im Skript stehen. Dies kann mit dem Target(bereich) beschrieben werden. Es ist ein Code aus ein oder zwei Buchstaben:

- a Es ist nur Material für eine sogenannte Langform, also ein Skript oder ein Buch in dem Verzeichnis enthalten.
- b Das Verzeichnis enthält Material für eine Präsentation.

Ein zusätzlicher zweiter Buchstabe kann es noch weiter einschränken. Beispielsweise steht „bh“ für ausschließlich Handouts, es werden also keine Slides erstellt.

⁴Ältere Programmierer kennen dieses Prinzip vielleicht noch von Programmiersprachen wie BASIC.

Ohne eine Target-Spezifikation enthält das Verzeichnis Material für *alle* konfigurierten Dokumentarten.

Dieser Zielbereich ist erweiterbar, siehe die Dokumentation zu `ollm`.

role Rolle, die die Einheit einnimmt. Dies wird stets durch einen einzelnen Buchstaben beschrieben:

- a Die Einheit ist ein Anhang.
- c Dies ist synonym zu einer ausgelassenen Rollenangabe und steht für eine normale Einheit.
- e Die Einheit ist ein Ergänzung. Damit ist gemeint, dass sie zur vorhergehenden Einheit zugehörig ist, aber eine gewisse Eigenständigkeit besitzt. Beispielsweise könnte es ein Übungsblatt oder Ergänzungsmaterial sein.
- i Diese Einheit integriert andere Einheiten. Beispielsweise würde die Integrationseinheit eines Vorlesungsskripts alle Skriptkapitel (die anderen Einheiten) zusammenfügen und ein Titelblatt, gemeinsames Inhaltsverzeichnis etc. hinzufügen. Jede Dokumentenart darf nur eine Integrationseinheit haben.

Bitte beachten sie, dass der Aufbau der Verzeichnisnamen nicht nur Konvention ist: Sowohl die Hauptklasse `osglecture.cls` als auch das Tool `ollm` werten die Namen aus.